

Hypothesis Testing in DEP: Design Matrix, Empirical Bayes, and FDR

Audience: Data scientists working with label-free quantification (LFQ) proteomics data using the DEP Bioconductor package.

Version context: DEP 1.32.0, limma 3.50, R 4.x

Reuse: This document is design-agnostic in its explanations; only the code examples reference the CTRL/KO/WT experiment from this repository. Replace those with your own conditions and comparisons.

Table of Contents

1. The Statistical Pipeline at a Glance
 2. Empirical Bayes Moderation (limma)
 3. Design Matrix: `~0 + condition` vs `~condition`
 4. The `formula` Argument in `test_diff()`
 5. FDR Concepts: BH and Storey's q-value
 6. FDR Options in DEP and How to Override
 7. Which FDR Method Is Best for Your Design?
 8. Code Inconsistency in This Repository
 9. Implementing Storey's q-value as a Drop-in Extension
 10. Decision Checklist
-

1. The Statistical Pipeline at a Glance

When you call `DEP::test_diff()`, the following sequence happens internally:

SummarizedExperiment (normalized, imputed intensities)

design matrix ← built from design_formula

limma::lmFit() per-protein linear model

limma::makeContrasts() contrast matrix from "A_vs_B" strings
limma::contrasts.fit() re-parameterize to contrasts

limma::eBayes() empirical Bayes variance shrinkage

limma::topTable(extract results per contrast
 adjust.method = "BH" ← BH FDR correction, hardcoded in DEP
)

```

add_rejections(          apply alpha and lfc thresholds
    alpha, lfc
)

```

Each step is described in detail in the sections below.

2. Empirical Bayes Moderation (limma)

¹ ### The core problem limma solves

In a standard t-test, the variance estimate for each protein is computed from its own replicates only. With 4 replicates per condition (as in this experiment), variance estimates are **noisy** — a protein could look significant simply because its variance happened to be estimated very low by chance.

limma solves this with **empirical Bayes (EB) moderation**: instead of using each protein's own variance estimate in isolation, it borrows information across all proteins to produce a more stable estimate.

The mathematical model

For protein g in contrast k , the standard linear model is:

$$\hat{\beta}_{gk} \sim N(\beta_{gk}, \sigma_g^2(X^T X)^{-1})$$

where β_{gk} is the true log2 fold-change, and σ_g^2 is the protein-specific residual variance.

The **empirical Bayes prior** assumes that the inverse variances follow a scaled chi-squared distribution:

$$\frac{1}{\sigma_g^2} \sim \frac{1}{d_0 s_0^2} \chi_{d_0}^2$$

where d_0 and s_0^2 are estimated from the data across all proteins. This yields the **moderated variance**:

$$\tilde{s}_g^2 = \frac{d_0 s_0^2 + d_g s_g^2}{d_0 + d_g}$$

The moderated variance $\tilde{s}_g^2$ is a weighted average between the **global prior variance** s_0^2 (shared information from all proteins) and the **protein-specific variance** s_g^2 (measured from its own replicates), weighted by their respective degrees of freedom.

The moderated t-statistic then uses $\tilde{s}_g^2$ instead of s_g^2 :

$$\tilde{t}_{gk} = \frac{\hat{\beta}_{gk}}{\tilde{s}_g \sqrt{v_k}}$$

This follows a t-distribution with $d_0 + d_g$ degrees of freedom (more degrees of freedom than a standard t-test → less extreme p-values for poorly estimated proteins).

What this means in practice

¹[²]

Situation	Without EB	With EB
Protein with low variance by chance	Falsely significant	Pulled toward global variance — less extreme
Protein with high variance by chance	Falsely non-significant	Pulled toward global variance — less conservative
Protein with many NAs, few replicates	Very unstable	Strongly borrows from global prior
Large experiment (many replicates)	Both converge	Borrowing effect diminishes — both agree

With only 4 replicates per condition (as in this repository), EB moderation provides a substantial stability benefit. The prior is estimated across all proteins quantified (~thousands in a typical LFQ experiment), so the estimates are robust.

3. Design Matrix: `~0 + condition` vs `~condition`

The design matrix encodes which samples belong to which experimental group. The choice of parameterization changes how contrasts are specified, **not** the final statistical results for the same comparison.

`~condition` — Treatment-contrast (reference-level) parameterization

This is R's default when a factor is included in a formula with an intercept.

Assuming `condition` levels are CTRL, KO, WT (alphabetical → CTRL is reference):

Coefficient	Meaning
(Intercept)	mean of CTRL (reference level)
<code>conditionKO</code>	KO – CTRL
<code>conditionWT</code>	WT – CTRL

Design matrix (12 samples, 4 per group):

	Intercept	<code>conditionKO</code>	<code>conditionWT</code>
CTRL_1	1	0	0
CTRL_2	1	0	0
CTRL_3	1	0	0
CTRL_4	1	0	0
KO_1	1	1	0
KO_2	1	1	0
KO_3	1	1	0
KO_4	1	1	0
WT_1	1	0	1
WT_2	1	0	1
WT_3	1	0	1
WT_4	1	0	1

To test KO_vs_WT you must construct the contrast `conditionKO - conditionWT`, which is less intuitive. Use `type = "control"` in `test_diff()` when adopting this parameterization.

`~0 + condition` — Cell-means (no-intercept) parameterization

Removing the intercept with 0 or -1 gives each level its own coefficient equal to the group mean:

Coefficient	Meaning
<code>conditionCTRL</code>	mean of CTRL
<code>conditionKO</code>	mean of KO
<code>conditionWT</code>	mean of WT

Design matrix:

	<code>conditionCTRL</code>	<code>conditionKO</code>	<code>conditionWT</code>
CTRL_1	1	0	0
CTRL_2	1	0	0
CTRL_3	1	0	0
CTRL_4	1	0	0
KO_1	0	1	0
KO_2	0	1	0
KO_3	0	1	0
KO_4	0	1	0
WT_1	0	0	1
WT_2	0	0	1
WT_3	0	0	1
WT_4	0	0	1

Each contrast maps directly to a difference of group means:

- `CTRL_vs_WT` → `conditionCTRL - conditionWT`
- `CTRL_vs_KO` → `conditionCTRL - conditionKO`
- `KO_vs_WT` → `conditionKO - conditionWT`

This is why `type = "manual"` and `~0 + condition` work naturally together.

Which to choose?

Scenario	Recommended parameterization
You want all pairwise comparisons	<code>~0 + condition + type = "manual"</code>
You have a clear reference group and compare everything to it	<code>~condition + type = "control"</code>
You want DEP to auto-generate all pairwise contrasts	<code>~0 + condition + type = "all"</code>
You want DEP to auto-generate comparisons to one control	<code>~condition + type = "control" + control = "CTRL"</code>

This repository uses `~0 + condition + type = "manual"` + explicit comparisons vector → correct choice for the experimental design.

4. The formula Argument in `test_diff()`

Function signature

```
test_diff(
  se,                                # SummarizedExperiment (input data)
  type,                              # "control" | "all" | "manual"
  control = NULL,                    # reference group (only for type="control")
  test = NULL,                       # contrast strings (only for type="manual")
  design_formula = formula(~0 + condition) # DEFAULT: cell-means
)
```

Default: `design_formula = formula(~0 + condition)` — so if you omit the argument, DEP uses cell-means parameterization.

How the formula affects hypothesis testing step by step

1. **Design matrix construction:** `model.matrix(design_formula, data = colData(se))` produces the numeric matrix that encodes your experimental groups.
2. **Contrast generation:** depending on `type`:
 - "manual": DEP parses each string in `test` (e.g. "CTRL_vs_WT") by splitting on "_vs_" and builds a contrast vector `conditionCTRL - conditionWT`.
 - "control": DEP generates one contrast per non-reference group.
 - "all": DEP generates all pairwise contrasts.
3. **Model fitting:** `lmFit(assay(se), design)` — the design matrix determines which samples contribute to each group mean coefficient.
4. **Contrast testing:** `contrasts.fit()` re-parameterizes the model to the specified contrasts, then `eBayes()` applies moderation.

What variables can appear in the formula?

The formula can include any column from `colData(se)`. Most commonly:

Formula	Effect
<code>~0 + condition</code>	Group means only (most common)
<code>~0 + condition + batch</code>	Correct for batch effects (one continuous or categorical covariate)
<code>~0 + condition + sex</code>	Correct for sex as a covariate
<code>~0 + condition + replicate</code>	Block on replicate if it is a technical/biological blocking factor

Warning: only add covariates to the formula if they represent a real source of variation you want to remove. Adding uninformative covariates reduces degrees of freedom and statistical power.

How to select the formula for this experiment

The experimental design is: - 3 conditions: CTRL, KO, WT - 4 replicates each (n = 12 total) - No batch variables recorded - Replicates are independent biological replicates

→ Correct formula: `formula(~0 + condition)` — as used in `04_data_analysis.R`.

If a batch variable were present (e.g., samples processed on two different days), you would extend it:

```
# hypothetical batch correction
design_formula = formula(~0 + condition + batch)
```

5. FDR Concepts: BH and Storey's q-value

The multiple testing problem

When testing thousands of proteins simultaneously, false positives accumulate. If you apply a nominal p-value threshold of 0.05 to 3000 proteins, you expect ~150 false positives by chance alone under the null.

False Discovery Rate (FDR) is the expected proportion of false positives among all rejected tests:

$$FDR = E \left[\frac{V}{R} \right]$$

where V = false positives, R = total rejections. FDR control is less stringent than Bonferroni (which controls the probability of *any* false positive), making it appropriate for exploratory biological discovery.

Benjamini-Hochberg (BH, 1995)

BH is the standard FDR correction. Given m tests with p-values $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$, the BH procedure rejects all $H_{(i)}$ where:

$$p_{(i)} \leq \frac{i \cdot \alpha}{m}$$

The BH-adjusted p-value for rank i is:

$$p_{(i)}^{adj} = \min_{j \geq i} \left(\frac{m}{j} \cdot p_{(j)} \right)$$

Key assumption: BH controls FDR at level α assuming $\pi_0 = 1$, meaning it assumes **all** null hypotheses are true. This is **conservative** when many proteins are truly differentially expressed (i.e., $\pi_0 < 1$).

Storey's q-value (2002)

The q-value method improves on BH by estimating π_0 — the proportion of truly null hypotheses — directly from the observed p-value distribution.

$$q_i = \frac{\hat{\pi}_0 \cdot m \cdot p_i}{|\{j : p_j \leq p_i\}|}$$

The intuition: if you look at p-values near 1 (where almost all are null), their density tells you $\hat{\pi}_0$. If $\hat{\pi}_0 = 0.8$, then only 80% of the tests are null, and the BH correction (which assumed 100%) is too conservative.

Comparison:

Property	BH	Storey q-value
Controls FDR?	Yes, at exactly α	Yes, adaptively
Assumes π_0	1 (conservative)	Estimated from data
Power when $\pi_0 < 1$	Lower	Higher
Power when $\pi_0 = 1$	Identical	Identical
Requires p-value distribution	No	Yes (should be uniform under null)
Risk	None (conservative)	Unstable if few tests or non-uniform null

Visual intuition: p-value histogram

Count

← excess at low p-values = true positives

← flat part null distribution
p-value
0 0.2 0.4 0.6 0.8 1.0

- The flat part at high p-values estimates $\hat{\pi}_0 \cdot m$ (expected null count).
- Storey uses this to calculate how conservative BH is being.

When does the choice matter most?

The difference between BH and q-value is largest when: - Many proteins are truly DE (e.g., global treatment effect, strong KO phenotype) - Sample sizes are small (so power is a concern) - The number of tests is large (thousands of proteins → larger multiple testing burden)

Both methods converge when $\pi_0 \approx 1$ (very few true positives) or when sample sizes are large.

6. FDR Options in DEP and How to Override

What DEP exposes

DEP 1.32.0 does **not** expose a `method` argument for FDR correction. The BH method is hardcoded inside `test_diff()` via:

```
# DEP source - simplified
limma::topTable(fit, coef = contrast, adjust.method = "BH", number = Inf)
```

The `add_rejections()` function then simply applies thresholds to the `p.adj` column that `test_diff()` already computed:

```
add_rejections(dep, alpha = 0.05, lfc = 1)
# marks as significant: p.adj < alpha AND abs(ratio) > lfc
```

There is no FDR method argument in `add_rejections()` either.

Available strategies to use alternative FDR methods

Strategy A: Post-hoc q-value on extracted p-values (recommended) Run DEP normally to get results, then replace or supplement `p.adj` with q-values:

```
library(qvalue)

results <- get_results(dep_analysis)

# Apply q-value per comparison
for (comp in comparisons) {
  pval_col <- paste0(comp, "_p.val")
  qval_col <- paste0(comp, "_q.val")

  pvals <- results[[pval_col]]

  # qvalue() requires all p-values; NA causes failure
  valid <- !is.na(pvals)
```

```

q_out <- qvalue(pvals[valid])

results[[qval_col]] <- NA_real_
results[[qval_col]][valid] <- q_out$qvalues
}

```

Then filter on q.val instead of p.adj.

```

results <- get_results(dep_analysis)

for (comp in comparisons) {
  pval_col <- paste0(comp, "_p.val")
  adj_col <- paste0(comp, "_p.adj_holm") # example: Holm-Bonferroni
  results[[adj_col]] <- p.adjust(results[[pval_col]], method = "holm")
}

```

Strategy B: Apply p.adjust() with any method to the raw p-values Supported method values in p.adjust():

Method	Type	When to use
"BH"	FDR (Benjamini-Hochberg)	Default; balanced power/control
"BY"	FDR (Benjamini-Yekutieli)	When tests are positively dependent (correlated proteins)
"bonferroni"	FWER	Very few tests; strict error control required
"holm"	FWER	Step-down Bonferroni; slightly more powerful
"fdr"	Alias for "BH"	—
"none"	No correction	Only for visualization, never for publication

Strategy C: Extend add_rejections() to use q-values If you want to keep the DEP object structure, add a custom post-processing step:

```

add_qvalue_rejections <- function(dep, alpha = 0.05, lfc = 1, comparisons) {
  results_df <- get_results(dep)

  for (comp in comparisons) {
    pval_col <- paste0(comp, "_p.val")
    qval_col <- paste0(comp, "_q.val")
    sig_col <- paste0(comp, "_significant_qval")

    valid <- !is.na(results_df[[pval_col]])
    q_out <- qvalue::qvalue(results_df[[pval_col]][valid])

    results_df[[qval_col]] <- NA_real_
    results_df[[qval_col]][valid] <- q_out$qvalues
    results_df[[sig_col]] <- results_df[[qval_col]] < alpha &
      abs(results_df[[paste0(comp, "_ratio")]]) > lfc
  }
  return(results_df)
}

```

7. Which FDR Method Is Best for Your Design?

Characteristics of this experiment

Parameter	Value	Implication
Conditions	3 (CTRL, KO, WT)	3 independent pairwise contrasts
Replicates	4 per condition	Small $n \rightarrow$ EB moderation critical
Comparisons	3 (CTRL_vs_WT, CTRL_vs_KO, KO_vs_WT)	Independent tests per contrast
Proteins tested	~hundreds to thousands	Large multiple testing burden
Expected biology	KO likely has many DE proteins	π_0 likely $< 1 \rightarrow$ q-value gain possible

Recommendation

Primary analysis: Use **Storey's q-value** applied per comparison as the significance criterion for publication-level calls, with BH as the fallback for robustness checks.

Reasoning:

1. With thousands of proteins and a meaningful KO, π_0 is likely 0.7–0.9, not 1.0. BH overcorrects in this range.
2. With only 4 replicates, power is limited — recovering more true positives with q-value matters.
3. The q-value p-value distribution assumption holds because limma p-values are well-calibrated (the EB moderation improves the degrees of freedom estimate).

When to stick with BH:

- If the p-value histogram does not show a clear spike near zero (few true positives $\rightarrow \pi_0 \approx 1$, q-value gives no benefit).
- For the diagnostic comparison of imputation methods (`DE_analysis()` in `03_cleaning_data_mixed_imputation.R`) — BH is sufficient there because the goal is relative comparison, not absolute significance claims.
- When the number of tested proteins is small (< 200 per comparison after filtering).

Cross-comparison FDR: should you correct globally or per comparison?

DEP applies correction **per comparison** (each contrast separately). This is the standard approach in proteomics and is correct for your design, because:

- The three comparisons address different biological questions.
- Combining all p-values globally would make the correction overly conservative (you would penalize a CTRL_vs_KO result for tests belonging to KO_vs_WT).

Global FDR across all comparisons is appropriate only when you are pooling tests that are exchangeable (same null hypothesis across comparisons) — not the case here.

8. Code Inconsistency in This Repository

What was found

The `DE_analysis()` helper in `03_cleaning_data_mixed_imputation.R` calls `test_diff()` without `design_formula`:

```
# 03_cleaning_data_mixed_imputation.R, lines 779-784
DE_analysis <- function(se) {
  se %>%
    test_diff(., type = "manual", test = comparisons) %>% # ← no design_formula
    add_rejections(., alpha = p_val, lfc = log2(FC)) %>%
    get_results()
}
```

The main analysis in 04_data_analysis.R explicitly passes it:

```
dep_analysis <- analyze_dep(imputation_file,
  type      = "manual",
  control   = NULL,
  alpha     = p_val,
  lfc       = FC,
  test      = comparisons,
  design_formula = formula(~0 + condition)) # ← explicit
```

Is this a bug?

No — the behavior is identical. `test_diff()` defaults to `formula(~0 + condition)`, which matches what the main analysis passes explicitly. The results are the same.

However, the omission is a **readability and maintainability risk**: if the default ever changes, or if someone copies `DE_analysis()` into a context with different experimental variables in `colData`, the implicit formula may produce wrong results silently.

Recommended fix

Make the formula explicit in the diagnostic function. In 03_cleaning_data_mixed_imputation.R, line 780:

```
# Before
DE_analysis <- function(se) {
  se %>%
    test_diff(., type = "manual", test = comparisons) %>%
    add_rejections(., alpha = p_val, lfc = log2(FC)) %>%
    get_results()
}

# After - explicit design_formula matches the main analysis
DE_analysis <- function(se) {
  se %>%
    test_diff(., type = "manual", test = comparisons,
      design_formula = formula(~0 + condition)) %>%
    add_rejections(., alpha = p_val, lfc = log2(FC)) %>%
    get_results()
}
```

Also note: `lfc = log2(FC)` in the diagnostic function passes `log2(0.5) = -0.585` as the fold-change threshold, while `analyze_dep(..., lfc = FC)` passes 0.5 directly. The `add_rejections()` `lfc` argument is in log2 scale, but `analyze_dep()`'s `lfc` argument is also in log2 scale — both are consistent. However, `FC = 0.5` means ± 0.5 in log2, which corresponds to a fold-change of $\sim 1.41\times$, not $2\times$. Verify this threshold is intentional.

9. Implementing Storey's q-value as a Drop-in Extension

The following function can be added to `code/aux_functions.R` or called directly in the analysis pipeline. It extends the DEP results table with q-values per comparison.

```
# Requires: qvalue (Bioconductor)
# Input:  dep_analysis object from analyze_dep() or test_diff()
#         comparisons character vector
# Output: data.frame identical to get_results() but with added _q.val columns
#         and _significant_qval logical column
get_results_with_qvalue <- function(dep_analysis, comparisons, alpha = 0.05, lfc = 0.5) {

  if (!requireNamespace("qvalue", quietly = TRUE)) {
    stop("Install qvalue: BiocManager::install('qvalue')")
  }

  results <- DEP::get_results(dep_analysis)

  for (comp in comparisons) {
    pval_col <- paste0(comp, "_p.val")
    ratio_col <- paste0(comp, "_ratio")
    qval_col <- paste0(comp, "_q.val")
    sig_col <- paste0(comp, "_significant_qval")

    if (!pval_col %in% names(results)) {
      warning("Column not found: ", pval_col, " - skipping ", comp)
      next
    }

    pvals <- results[[pval_col]]
    valid <- !is.na(pvals)

    if (sum(valid) < 100) {
      warning(comp, ": fewer than 100 valid p-values - q-value estimate may be unreliable")
    }

    q_out <- tryCatch(
      qvalue::qvalue(pvals[valid]),
      error = function(e) {
        warning(comp, ": qvalue() failed (", e$message, ") - falling back to BH")
        list(qvalues = p.adjust(pvals[valid], method = "BH"))
      }
    )

    results[[qval_col]] <- NA_real_
    results[[qval_col]][valid] <- q_out$qvalues

    results[[sig_col]] <- !is.na(results[[qval_col]]) &
      results[[qval_col]] < alpha &
      abs(results[[ratio_col]]) > lfc
  }

  return(results)
}
```

Usage in 04_data_analysis.R

```
# After running data_analysis(), supplement with q-values:
results_qval <- get_results_with_qvalue(
  dep_analysis = dep_analysis,
  comparisons  = comparisons,
  alpha        = p_val,      # 0.05
  lfc          = FC          # 0.5 (log2 scale)
)

# Inspect estimates
library(qvalue)
for (comp in comparisons) {
  pvals <- results_qval[[paste0(comp, "_p.val")]]
  q_out <- qvalue(pvals[!is.na(pvals)])
  cat(comp, "→ estimate:", round(q_out$pi0, 3), "\n")
}

# Compare BH vs q-value significant counts
for (comp in comparisons) {
  n_bh <- sum(results_qval[[paste0(comp, "_significant")]], na.rm = TRUE)
  n_qv <- sum(results_qval[[paste0(comp, "_significant_qval")]], na.rm = TRUE)
  cat(comp, "→ BH:", n_bh, " | q-value:", n_qv, "\n")
}
```

10. Decision Checklist

- ☐ Before running the differential analysis, verify each item:
 - ☐ **Formula matches type:** ~0 + condition with type = "manual" OR ~condition with type = "control".
 - ☐ **Contrast strings match condition labels exactly:** Check `unique(colData(se)$condition)` and compare with your comparisons vector.
 - ☐ **design_formula is explicit** in all calls to `test_diff()` (not relying on the default).
 - ☐ **Batch effects:** If `colData(se)` has a batch variable, include it in the formula: ~0 + condition + batch.
 - ☐ **FDR method matches your claim:** Use BH for conservative exploratory analysis; use Storey's q-value when power is critical and $\pi_0 < 1$ is expected.
 - ☐ **FDR applied per comparison:** Do not pool p-values across comparisons before correction.
 - ☐ **Inspect p-value histogram** per comparison to verify the null distribution is uniform at high p-values (validates EB calibration and FDR assumptions).
 - ☐ **FC threshold units:** `add_rejections(lfc = ...)` expects log2 scale; `analyze_dep(lfc = ...)` also expects log2 scale. Do not exponentiate.
 - ☐ **significant vs significance:** In this codebase, `significant` (from `add_rejections`) requires *all* comparisons to be consistent (DEP's global flag); `significance` (added in 04_data_analysis.R) is TRUE if *any* comparison passes the nominal threshold. Use the appropriate one for your question.
-

References

- Smyth GK (2004). *Linear models and empirical Bayes methods for assessing differential expression in microarray experiments*. Statistical Applications in Genetics and Molecular Biology.
- Benjamini Y, Hochberg Y (1995). *Controlling the false discovery rate: a practical and powerful approach to multiple testing*. JRSS-B.
- Storey JD, Tibshirani R (2003). *Statistical significance for genome-wide studies*. PNAS.
- Zhang X et al. (2018). *Proteome-wide identification of ubiquitin interactions using UbIA-MS*. Nature Protocols. (DEP paper)
- DEP Bioconductor vignette
- limma User's Guide